

eBPF NFTracer — AMF Threat-Aware State Machine

Progress report · 2026-07-17 · module: amf/

Overview

Goal: instrument free5gc's AMF with eBPF uprobes so a userspace agent can reconstruct, in real time, entry into each of the 23 conceptual states of the threat-aware UE-registration FSM defined in `amf_state_machine.py / ebpf-hook-point.pdf`. Work this session moved the tracer from "instruments an unrelated API surface" to "instruments the verified FSM hook points, with the FSM itself stored as a real eBPF hash map." Five milestones, in order:

- **1. Gap analysis** — compared the PDF's 23-state hook-point spec against the existing tracer.
- **2. Legacy cleanup** — disabled the unrelated probe set that gap analysis found.
- **3. Binary verification** — confirmed all 40 target Go symbols actually exist in `bin/amf`.
- **4. FSM-as-eBPF-map** — built the 29-node/52-edge FSM as real `BPF_MAP_TYPE_HASH` maps (CO-RE).
- **5. Full instrumentation** — wired 43 verified uprobes across all four AMF tracer source files.

1. Gap Analysis

Read `ebpf-hook-point.pdf` (23-state FSM, Parts 1-2 states + Part 3 supplementary/edge hooks) and compared every cited Go symbol against `amf_functions.c`. Result: **zero overlap**. The existing tracer hooked 23 `internal/sbi/processor.(*Processor).*SBI-service` callbacks (Subscription, UE-Context, Callback, Event-Exposure, N1N2-Message, OAM APIs) — a completely different surface from the FSM's `internal/gmm`, `internal/ngap`, `internal/nas`, `internal/context`, and `internal/sbi/consumer` packages. The only PDF-listed hook already implemented anywhere was the XDP SCTP/NGAP PPID=60 ingress probe, in `xdp_ngap.c` (separate from the uprobe set).

Deliverable

`amf/doc/missing_attach_points.txt` — full list of all 23 states' hook points (Parts 1-3), each with type (UPROBE / URETPROBE / DERIVED), Go symbol, and source file:line, ready to drive implementation.

2. Legacy Probe Cleanup

Disabled (via `#if 0`, not deleted — reversible) the 23 SBI-processor probes identified as out-of-scope, across every file that referenced them:

- `amf_tracer.bpf.c` — all 23 `SEC("uprobe/amf_*")` programs.
- `amf_functions.h` — the six `attach_target[]` extern declarations.
- `amf_functions.c` — the six array definitions.
- `amf_loader.c` — the `attach_programs()` / `detach_programs()` call sites.

Verified consistency across all four files after disabling (no dangling references, matching `#if 0/#endif` pairs).

3. Binary Verification

Cross-checked all 40 distinct Go symbols cited across the PDF's 23 states + Part 3 against the actual bin/amf ELF (statically linked, not stripped, DWARF debug info present). Method: `nm bin/amf` for existence, `go tool objdump -s '<symbol>' bin/amf` for the DWARF-resolved source file:line, cross-checked against the PDF's cited location.

Result: 40/40 present

- Most symbols match the PDF's file:line exactly (e.g. `internal/nas.HandleNAS` at `handler.go:14`).
- ~12 entries differ by 1-20 lines — DWARF cites the first body instruction, PDF cites a multi-line signature start. Not wrong symbols.
- Two needed clarification: `internal/gmm.Authentication`'s body actually starts at `sm.go:150` (PDF's 222 is an inner switch-branch); `context.(*AmfUe).SecurityContextIsValid` is defined at `amf_ue.go:468` (PDF's `sm.go:272` is its call site).
- `ue.MacFailed` correctly has no symbol — it's a struct field, matching the PDF's own [DERIVED] tag.
- `free5gc/util/fsm.(*FSM).SendEvent` confirmed resolvable in the static binary despite living in an external module.

Deliverable

`amf/doc/verified_attach_points.txt` — all 40 symbols, full verbatim Go symbol strings ready to drop into `attach_target.func_name`, with binary-vs-PDF line-number footnotes.

4. FSM Stored as a Real eBPF Hash Map (CO-RE)

First pass built the 29-node/52-edge FSM as a userspace hash table (malloc'd, chained buckets, self-tested standalone). Refined into an actual in-kernel construct once it became clear the goal was live in-kernel lookups, not just a userspace data structure:

- `sm_map.h` — shared, fixed-size, pointer-free key/value structs (`sm_node_key/val`, `sm_edge_key/val`), valid unchanged on both the BPF and userspace side (same pattern as `events.h`'s `struct event`).
- `sm_map.c` — pure BPF/CO-RE source (built against `vmlinux.h`): two `BPF_MAP_TYPE_HASH` maps (`sm_nodes`, `sm_edges`) via `SEC(".maps")`, plus `__always_inline` in-kernel lookup helpers (`sm_node_lookup`, `sm_transition_lookup`) that a uprobe handler can call to validate a UE's state transition *before the event reaches userspace*.
- `amf_tracer.bpf.c` — `#include "sm_map.c"` wires the maps into the compiled BPF object.
- `amf_comm.c/.h` — `sm_map_populate(skel)`: since a BPF hash map has no static-initializer mechanism, this loads all 29 nodes / 52 edges via `bpf_map_update_elem()` at runtime.
- `amf_loader.c` — calls `sm_map_populate(skel)` right after skeleton load.

Verified

- Standalone userspace version: compiled clean, ran, printed all 29 nodes / 52 edges matching `amf_state_machine.py` exactly; `sm_transition("AUTH_VECTOR_PENDING", "AUSF success")` correctly resolved to `NAS_AUTHENTICATING`.
- BPF version: `clang -target bpf -fsyntax-only` against the real repo `vmlinux.h` — 0 errors, 0 warnings.

5. Full FSM Instrumentation — 21 Groups, 43 Uprobes

Built the production `attach_target[]` arrays from `verified_attach_points.txt`, one array per FSM node (states that add no new symbol — `REPLAY_SUSPECTED`, `PDU_SESSION_REJECTED` — get a comment pointing at the state whose probe they reuse, not a duplicate attach). All `prog_name` values kept ≤ 15 characters.

File	Change
<code>amf_functions.h</code>	21 new <code>attach_target[]</code> + count externs, grouped by FSM part
<code>amf_functions.c</code>	21 new arrays, 43 <code>attach_target</code> entries, verified <code>func_name</code> strings
<code>amf_tracer.bpf.c</code>	43 new <code>SEC("uprobe/...")</code> handlers, grouped/commented to mirror the arrays
<code>amf_loader.c</code>	<code>attach_programs()/detach_programs()</code> wired for all 21 groups

Verification performed (not just written)

- Every `prog_name` ≤ 15 chars, checked programmatically — 0 violations.
- Every `prog_name` unique — 0 duplicates.
- Every `__builtin_memcpy(e->api, "...", N)` length checked against the actual string length — **caught and fixed two off-by-one bugs** (`REG:NGAPInitialUEMsg`, `CTXLOOKUP:XferOldAmf`, both miscounted 21 vs. actual 20) that would have silently corrupted ring-buffer events.

- Cross-checked `prog_name` ↔ `SEC( )` name ↔ C function name ↔ extern declaration ↔ attach/detach call site across all four files — 43/22 fully consistent, 0 missing or orphaned references.
- `gcc -fsyntax-only` on the userspace files and `clang -target bpf -fsyntax-only` on the full BPF file (against the real `vmLinux.h`) both compile clean.

Current State

The tracer now attaches 43 uprobes covering all 21 FSM-node groups (13 normal-path states, 8 failure states that add a new symbol, plus Part 3 supplementary/edge hooks), backed by a real in-kernel BPF hash map holding the full 29-node/52-edge FSM. Each handler currently emits a simple `EVT_API_CALL` event tagged "`<STATE>:<Symbol>`"; the in-kernel `sm_transition_lookup()` primitive exists but is not yet called from any handler.

Build note: the checked-in `amf_tracer.skel.h / .o` files predate these changes — run `make clean && make` on the build machine to regenerate before deploying.

Suggested Next Steps

- Wire `sm_transition_lookup()` into the uprobe handlers so out-of-spec transitions are flagged in-kernel, not just recorded.
- Deploy on the Linux build machine, run `make`, and validate live attachment against a running AMF container end-to-end.
- Extend `URETPROBE` handlers to classify return values (success / reject / timeout) per the PDF's `NF_TIMEOUT` derived-signal guidance, rather than recording a bare call event.
- Consider the still-optional packet-ingress Go-level hooks (`AcceptSCTP`, `SCTPRead`, `handleConnection`, `dispatchToWorkerPool`) noted in `verified_attach_points.txt` if finer-grained ingress timing is needed beyond the existing XDP probe.